CETP Handbook

Everything about this project, in reading order. Revision 2026-10-13 · Read this once and you will know the whole system.

Part I — What this is

1. The one-paragraph version

A developer runs a query on their laptop; it takes 400 ms. Production runs on different hardware. CETP predicts how long that query will take **there**, from a plan captured **here**. Within one CPU architecture it does this at **29.31% error against 37.39%** for simply assuming the two machines behave alike. Across CPU architectures it does not work without calibration, and the system refuses rather than guessing.

2. What makes it interesting

Not the predictor. The interesting part is a finding that generalises beyond this project:

> **Encoding hardware capability as a measured number works within a CPU architecture and fails across one.** Three independent measurements say so, each with the mechanism identified. Calibration is not an optimisation — across an architecture boundary it is a precondition.

That indicts a whole class of prior work, including the hardware-feature vectors used by learned cost models.

3. Vocabulary

Term	Meaning
dev / prod	The machine you measure on, and the machine you want to predict
MAPE	Mean absolute percentage error. 20% means predictions are typically 20% off
naive baseline	"Assume production behaves like development." The bar to beat
scaling factor	prod runtime ÷ dev runtime. What the model actually predicts
capability signature	Numbers describing how fast a machine is at various operations
LOMO	Leave-one-machine-out: train on all machines but one, test on it
ISA	Instruction set architecture — arm64 (Apple Silicon) vs x86_64 (Intel/AMD)
DOP	Degree of parallelism. In PostgreSQL, `1 + max_parallel_workers_per_gather`
q-error	`max(pred/actual, actual/pred)`. 1.0 is perfect. The database literature's metric
calibration	One multiplicative correction per machine pair, learned from measurements
elasticity	Exponent `e` in `T(N) = T(1)·N^e` describing slowdown under N clients

---

Part II — How it works

4. The pipeline

```
1. MEASURE CAPABILITY     operator_probe.py runs six operations in SQL on each machine
2. CAPTURE A PLAN         EXPLAIN (ANALYZE) on dev; EXPLAIN (no ANALYZE) on prod
3. REDACT                 strip literals before the plan leaves the machine
4. DECOMPOSE              split the plan into per-node self time
5. SCALE                  weight each operator by dev's time share, apply capability ratios
6. ADJUST                 parallelism, cardinality, concurrency, calibration
7. REPORT                 predicted runtime + trust level + every applicable warning
```

5. Step 1 — the capability probe

```
python3 scripts/collection/operator_probe.py --machine-id my-laptop --db mydb
```

Creates two throwaway tables, runs six operations, records tuples/s, drops the tables. Takes a few minutes. **Run it on an idle machine** — under load it measures contention, and the tool flags the record unreliable if it detects that.

Axis	What it measures
`seqscan`	Scanning and filtering rows
`hashjoin`	Building and probing a hash table
`sort`	Sorting
`hashagg`	Hash aggregation (GROUP BY)
`nestloop`	Indexed lookups — random access and TLB behaviour
`parallel_seqscan`	The same scan with parallelism on → `parallel_speedup`

Why SQL and not a C benchmark? Three reasons, all measured:

1. A C probe needs a shell and a compiler, so it cannot describe **RDS, Aurora or Cloud SQL** — where most production PostgreSQL lives.
2. The old C probe measured floating-point throughput. **No database query issues those instructions.**
3. It scored 324% error laptop→cloud, worse than doing nothing.

6. Step 4 — decomposing a plan

PostgreSQL's `Actual Total Time` has two traps:

It is cumulative. A node's reported time includes all its children. Self time = inclusive − children.

It is per-loop. An inner node reports the *average* of one loop. An index scan showing `0.31 ms` over `485 loops` actually cost **150 ms**. Multiply by `Actual Loops`.

And a third, specific to parallelism. Everything below a `Gather` runs once per worker, but only nodes that *partition* work carry `Parallel Aware`. Keying on that flag instead of "below a Gather" corrects 46% of a plan instead of 100%:

Keyed on	Predicted	True
nothing	1.080	3.011
`Parallel Aware`	1.493	3.011
below a `Gather`	**2.664**	**3.011**

7. Step 5 — the formula

```
scaling = Σ  w_op · [rate_dev(op) · s_dev] / [rate_prod(op) · s_prod] · card_op
T_prod  = T_dev × scaling × calibration × N^e
```

`w_op` is dev's **wall-time** share for that operator. The weights sum to 1.

Why a weighted ratio rather than summing scaled node times? Under a `Gather`, node times sum to total *CPU* time — measured at **2.85× wall** on one query. Summing would predict ~3× too high. Normalising to weights cancels that exactly.

8. Step 7 — trust levels

Trust	When
`HIGH`	Production's plan was forced on dev and verified, and a calibration exists
`MEDIUM`	Uncalibrated, or production's plan unavailable, or >25% of time unmapped
`LOW`	Plan forcing failed, <50% of time priced, or cross-architecture uncalibrated

Every prediction also carries warnings — single-client, cross-architecture, unmapped time, default elasticity. **They are not decoration.** A millisecond figure with no error attached invites more trust than the measurements support.

Part III — What the measurements say

9. The headline numbers

Experiment	naive	model	Source
Capability grid, 1 232 pairs	72.52%	**24.95%**	`results/operator_model.md`
Laptop → 7 arm64 machines, 154 pairs	37.39%	**29.31%**	`results/laptop_to_prod.md`
Laptop → 4 x86 machines, 88 pairs	**21.74%**	68.92%	`results/cross_isa_prediction.md`
…the same, after 5 calibration queries	21.41%	**23.38%**	same
Legacy 5-machine EC2 dataset	23.05%	21.61%	`results/CANONICAL.md`

10. The five findings worth knowing

① Parallelism is the result; per-operator detail is not. Remove the parallelism term and the model is level with doing nothing (76.21% vs 72.52%). Localise it to the plan's parallel part and you gain **+5.32 pp** (CI [+0.14, +12.04]). Weight per operator on top of that and you gain **−0.37 pp** (CI [−0.84, +0.01]) — nothing. **A single capability scalar, correctly localised, matches the full model.**

② The probe does not cross architectures, and not for the reason expected. We expected the ISA to change the *relative* cost of operators. It does not: arm64→x86 perturbs the mix by **1.365×**, while the worst pair *within one host* perturbs it by **1.371×**. What fails is the **absolute scale** — the probe said x86 was 1.667× slower when it was 1.044×.

③ Calibration repairs it, and only to parity. Five production queries take the cross-ISA case from 72.84% to 23.38%, against a floor of 16.75%. But naive on that pair is 21.41%, so the capability model contributes **nothing** there that calibration did not.

④ A hard ceiling exists on this entire model class. Any predictor that multiplies by one number per machine pair is bounded by how much the true scaling varies *across queries on one machine*. On the EC2 data that is 18–43×, giving a **59.7%** MAPE floor. The best measurement reached 55.88% — at the floor.

⑤ Concurrency cannot be predicted from a plan. Leave-one-query-out, plan features score MAE **0.191** against **0.167** for just using the median. A global default halves the error at 8 clients (73.79% → 41.81%); knowing the query's own elasticity would reach 6.72%. So load is a caller input.

11. What "improves with use" means, precisely

Accuracy improves **for an environment pair** as measured observations of that pair accumulate:

Observations on the target	cross-ISA MAPE
0	72.84%
3	23.86%
5	23.38%

It does **not** improve with the number of machines in the registry. A correction absorbs *that machine's* scale error and says nothing about another. Claiming otherwise would need pooling across machines, which has not been demonstrated here.

Also: one observation is worse than none. A single arbitrary measurement scored 19.51% against 15.84% uncalibrated. Corrections below 3 observations are marked *provisional*.

Part IV — The repository

12. Map

```
cetp/                     the installable package (no runtime dependencies)
  plan_nodes.py           plan → per-node self time; node type → operator axis
  operator_model.py       the scaling formula, calibration fitting, concurrency
  cross_env.py            orchestration, trust, warnings, cross-arch refusal
  calibration_store.py    per-pair correction state
  telemetry.py            predicted-vs-actual logging
  ingest.py               auto_explain + pg_stat_statements
  redact.py               strip literals; verify the strip
  predict.py regress.py signature.py    legacy two-scalar path
  cli.py                  the `cetp` command

scripts/
  collection/
    operator_probe.py     THE capability probe
    predict_prod.py       end-to-end over two live connections
    container_grid.py     orthogonal CPU×memory capability grid via Docker
    collect.py            the measurement harness
    plan_to_hints.py      plan → pg_hint_plan hints (plan forcing)
    hw_signature.py       legacy two-scalar signature
    queries/ queries_tpcds/ queries_job/     22 + 99 + 113 queries
  analysis/
    canonical_evaluation.py        THE source of truth for the legacy dataset
    operator_model_evaluation.py   the capability grid
    laptop_to_prod_evaluation.py   the real scenario
    laptop_to_ec2_evaluation.py    cross-ISA on real cloud hardware
    concurrency_model.py           elasticity, fitted and cross-validated
    version_compat_check.py        parser vs PostgreSQL 14/15/17/18
  docs/build_pdf.py       Markdown → PDF

results/                  every number, generated. Never edited by hand
models/                   shipped artifacts + MODEL_CARD + PROVENANCE
data/                     datasets (large inputs are gitignored)
site/index.html           the project page, one self-contained file
docs/                     LLD.md/.pdf and this handbook
vscode-extension/         the editor integration
gate/                     the SLA gate and its CI demo
tests/                    108 tests
```

13. The documents, and which to trust

Document	Status
`results/*.md` + `*.json`	**Generated. Authoritative.**
`docs/LLD.md` / `.pdf`	Current design. Supersedes the old LLD PDF entirely
`docs/HANDBOOK.md`	This file
`site/index.html`	Public summary, same numbers
`OPERATOR_MODEL.md`	Design and status of the per-operator work
`TASKS.md`	Full history: what was done, what failed, what is open
`DOC_CORRECTIONS.md`	Itemised errors in the old deck/LLD
`models/MODEL_CARD.md`	What the shipped artifacts are and are not
Old LLD PDF / Review-2 deck	**Superseded. Do not quote.**

Part V — Running it

14. Install and smoke-test

```
pip install -e .
cetp selftest
```

15. Predict a query's production runtime

```
# once per machine, on an IDLE machine
python3 scripts/collection/operator_probe.py --machine-id dev-laptop --db mydb
python3 scripts/collection/operator_probe.py --machine-id prod-db --db mydb \
    --host prod.example.com --user readonly

# per query
python3 scripts/collection/predict_prod.py --query-file q.sql \
    --dev dev-laptop --prod prod-db \
    --prod-host prod.example.com --prod-db mydb --prod-user readonly \
    --dev-db mydb --sla-ms 2000
```

Production is only `EXPLAIN` ed — never executed. Its plan is then forced on dev with `pg_hint_plan` and measured there, so plan divergence cannot contribute.

16. Regression checking (the most reliable mode)

```
cetp regress --before before.json --after after.json --dev c7i --prod c5a \
             --threshold 1.20 --fail-on-regression
```

The hardware term appears in numerator and denominator and largely cancels, so the answer is essentially your measured dev-side ratio. This is the mode to use in CI.

17. Regenerating every published number

```
make results            # the legacy 5-machine dataset
make probe-here         # this machine's capability signature
make grid-setup         # load TPC-H SF1 into a Docker volume (once, ~10 min)
make grid-collect       # sweep the capability grid (~2-3 h)
make grid-reprobe       # re-probe cells without recollecting
make operator-results   # the capability grid evaluation
make verify-determinism # run twice, assert byte-identical
make test               # 108 tests

python3 scripts/analysis/laptop_to_prod_evaluation.py
python3 scripts/analysis/laptop_to_ec2_evaluation.py
python3 scripts/analysis/concurrency_model.py
python3 scripts/analysis/version_compat_check.py
python3 scripts/docs/build_pdf.py docs/LLD.md
```

18. Collecting a new machine

Any machine you can reach with `psql`. For a free x86 machine, the GitHub Actions workflow `.github/workflows/x86-operator-probe.yml` loads TPC-H, probes, collects, and uploads the results as an artifact.

Part VI — History: what went wrong and how it was found

This section exists because the mistakes are the most instructive part of the project, and because a reader who does not know them will misread the numbers.

19. Dataset and signal bugs

Bug	Effect	How it was caught
Buffer counters double-counted	Whole dataset wrong	Audit; headline survived at 21.61% vs 23.05%
Compute probed at `nproc`, not achievable DOP	324% → 153% laptop→cloud	A query can use only `1 + max_parallel_workers_per_gather` processes
First repetition is a cold-cache run	All p95/p99 contaminated	Rep 1 was slowest in 61 of 105 cells, up to 8.33×
Classifier leaked across CV folds	Inflated accuracy	Refit inside every fold
Shipped gate used single-core scores	Gate did not implement the headline	Now verified at runtime, not asserted

20. Measurement-hygiene failures — the most important lesson

Published numbers were corrected four times, every time because of the instrument.

1. **The reliability gate checked the wrong statistic.** It gated on run-to-run spread while the reported figure is a *median*. Parallel measurements jitter wildly (cv 0.587) while their median reproduces to 3%. The gate rejected half the good machines, shifted the population between runs, and moved headline numbers 5–15 points. Now gates on the bootstrap standard error of the median.
2. **Probe runs were 20× too short.** An 18 ms parallel scan is mostly worker startup; one machine reported an *impossible* 1.81× speedup on a single CPU. At realistic size it reports 0.808× — correctly slower.
3. **Probing a busy machine** moved its measured speedup from 2.47× to 1.14×.
4. **A storage axis was built and does not measure storage** — it tracks CPU quota (+0.762) over memory (+0.548). Kept, documented, unused.

21. A fabricated result, caught internally

An early causal experiment reported a **1.54× "plan effect" measured over cells where the plan was identical** — impossible. Two silent bugs: `pg_hint_plan` was never `LOAD`ed, so nothing was ever actually forced (verification passed on 100% of identical-plan cells and **0%** of the ones that mattered); and the paired arms were measured in separate sweeps, so cache state drifted — one query showed 13,304 ms against 539 ms for the *same plan*.

Both fixed; the corrected experiment **reversed the earlier conclusion**, putting plan divergence at **70.5%** of the cross-environment difference when the plan changes.

22. A security bug found by building the product loop

`auto_explain` emits a `Query Text` field containing the whole statement with every literal. **The redactor passed it through untouched** — nothing had exercised that path, because hand-captured `EXPLAIN` output omits the field. Now redacted, dropped entirely at `--level full`, and checked by `assert_clean`.

Part VII — Limits and honest claims

23. What this cannot do

1. **Cross-architecture prediction without calibration.** Refused by design.
2. **Beat the naive baseline across architectures**, even calibrated. It reaches parity.
3. **Differing data volumes.** The cardinality term has never been exercised above a ratio of 1.0.
4. **Storage differences.** No probe axis measures them. This is the diagnosed residual.
5. **Concurrency.** An input, not a prediction.
6. **Replace a staging benchmark** when an SLA decision rests on an absolute number.

24. The evaluation's own boundaries

22 TPC-H queries, scale factor 1, single client, one engine version per environment. Environments are 8 containers on one arm64 host plus 4 AMD CI runners and 5 historical EC2 instances — **not a production fleet**. PostgreSQL 14, 15, 17, 18 tested; **13 and 16 are not**.

25. Claims you can defend

> CETP predicts PostgreSQL query runtime on an unseen production machine from a plan captured on a developer's machine, at **29.31% MAPE against 37.39%** for assuming the machines are identical, over 13 environments and 242 query-machine pairs spanning two instruction sets. Accuracy improves per environment pair as measured observations accumulate (72.84% → 23.38% on five). Across an architecture boundary the capability signature does not transfer, and the system requires calibration there rather than returning an unreliable estimate.

26. Claims you cannot

- "Accurate to X%" without saying *same-architecture* and *single-client*.
- "Gets better as it sees more machines." It gets better per *pair*.
- "Per-operator modelling is the contribution." The ablation says otherwise.
- "Generalises across hardware." Across 8 containers on one host, 4 AMD runners and 5 EC2 types, all at SF1.
- Anything from the old LLD PDF or the Review-2 deck.

Part VIII — Reference

27. Where each number lives

Topic	File
Legacy 5-machine dataset	`results/CANONICAL.md`, `canonical_metrics.json`
Capability grid + ablations	`results/operator_model.md`
Laptop → production	`results/laptop_to_prod.md`
Cross-ISA prediction	`results/cross_isa_prediction.md`
Why it is not operator mix	`results/cross_isa_operator_rates.md`
The 59.7% ceiling	`results/laptop_to_ec2.md`
Concurrency	`results/concurrency.md`
PostgreSQL version compatibility	`results/version_compat.json`
Causal plan-drift decomposition	`results/tier4_plan_drift_causal.md`

28. Open work

Before presenting: the old deck still contains the fabricated Appendix C/D content.

Cheap and valuable: a second x86 *vendor* (all four are AMD); a non-CI host; comparison against a learned cost model on the same data (the only baselines today are self-ablations).

Blocked on ~40 GB of disk: a real I/O axis (needs a probe table larger than RAM) and SF10 (the only way to test differing data volumes *and* plan divergence).

Product, not research: per-repo multi-tenancy beyond file isolation; PostgreSQL 13/16 compatibility; MySQL or DuckDB.

29. Frequently asked

Why is the naive baseline so good on x86? Because that machine happens to run this workload at nearly the same speed as the laptop (true scaling 1.044×). When two machines are genuinely alike, "assume they're alike" is an excellent predictor — and a capability model can only lose. The model earns its keep when they differ, which is why it wins by 39 points on the 1-CPU machines.

Why not just use a learned model? On the legacy dataset the learned model scores 17.29% when it has seen the target machine and 26.86% when it has not — it interpolates across hardware and does not extrapolate. The analytical formula is protocol-invariant (21.61 / 21.66 / 21.64).

Is the bottleneck classifier useful? No. Macro-F1 0.375 with `io` recall **0.000**, and a flat uniform weighting scores 21.55% against its 21.61%. It is retained for continuity and the installable package does not use it.

Why is `cetp predict` different from `cetp predict-prod`? `predict` is the legacy two-scalar path the VS Code extension still uses. `predict-prod` is the per-operator path with the parallelism term, calibration and the cross-architecture guard. **Prefer `predict-prod`.**